

PICO Checkweigher System - Complete Documentation

Date: February 1, 2026

Project: TCP/IP Data Collection → PostgreSQL → Appsmith Dashboard

Table of Contents

1. [System Overview](#)
 2. [Architecture](#)
 3. [Installation & Setup](#)
 4. [Node-RED Flows](#)
 5. [PostgreSQL Database](#)
 6. [Appsmith Dashboard](#)
 7. [Issues Faced & Solutions](#)
 8. [Testing & Verification](#)
-

System Overview

A real-time data collection and visualization system that:

- Receives sensor data from TCP/IP devices
- Stores data in PostgreSQL database
- Displays live dashboard in Appsmith with auto-refresh

Data Flow:

TCP Device → Node-RED → PostgreSQL → Appsmith Dashboard

Architecture

Technology Stack

Component	Technology	Port	Purpose
Database	PostgreSQL 16	5432	Data storage
Data Pipeline	Node-RED	1880	TCP parsing & DB insert
Dashboard	Appsmith CE	8080	Data visualization
DB Admin	pgAdmin 4	5050	Database management

Docker Network

All services run in Docker containers on the same network (`app_network`):

- Containers communicate using **container names** (not IPs)
- Example: `postgres_db`, `node_red`, `appsmith`

Installation & Setup

Prerequisites

- Docker & Docker Compose installed
- WSL2 (for Windows users)

Step 1: Create Project Structure

```
project/
├── docker-compose.yml
└── init.sql
```

Step 2: Docker Compose Configuration

docker-compose.yml:

```
yaml
```

services:

postgres:

image: postgres:16

container_name: postgres_db

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD: yourpassword

POSTGRES_DB: checkweigher

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data

- ./init.sql:/docker-entrypoint-initdb.d/init.sql

networks:

- app_network

pgadmin:

image: dpage/pgadmin4:latest

container_name: pgadmin

environment:

PGADMIN_DEFAULT_EMAIL: admin@admin.com

PGADMIN_DEFAULT_PASSWORD: admin

ports:

- "5050:80"

depends_on:

- postgres

networks:

- app_network

nodered:

image: nodered/node-red:latest

container_name: node_red

ports:

- "1880:1880"

volumes:

- nodered_data:/usr/local/share/.node-red

depends_on:

- postgres

networks:

- app_network

appsmith:

image: appsmith/appsmith-ce:latest

```
container_name: appsmith
ports:
  - "8080:80"
volumes:
  - appsmith_data:/appsmith/data
networks:
  - app_network
```

```
networks:
  app_network:
    driver: bridge
```

```
volumes:
  postgres_data:
  nodered_data:
  appsmith_data:
```

init.sql:

```
sql

CREATE TABLE IF NOT EXISTS sensor_data (
  id SERIAL PRIMARY KEY,
  product_name VARCHAR(255),
  product_count INTEGER,
  weight FLOAT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Step 3: Deploy

```
bash

docker compose up -d
```

Step 4: Verify Containers

```
bash

docker ps
```

Expected Output:

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
xxx	postgres:16	Up 2 minutes	0.0.0.0:5432->5432/tcp	postgres_db
xxx	nodered/node-red:latest	Up 2 minutes	0.0.0.0:1880->1880/tcp	node_red
xxx	appsmith/appsmith-ce	Up 2 minutes	0.0.0.0:8080->80/tcp	appsmith
xxx	dpage/pgadmin4:latest	Up 2 minutes	0.0.0.0:5050->80/tcp	pgadmin

Node-RED Flows

1. TCP to PostgreSQL Flow

Purpose: Receive TCP data, parse it, and insert into PostgreSQL

Flow Steps:

1. **TCP In** → Receives data from `192.168.2.12:5000`
2. **Parse TCP Data** → Handles JSON, CSV, or Buffer formats
3. **Build INSERT Query** → Creates SQL query
4. **PostgreSQL Insert** → Executes query
5. **Debug** → Logs results

Supported Data Formats:

- JSON: `{"product_name":"Widget","product_count":10,"weight":1.5}`
- CSV: `Widget,10,1.5`
- Binary/Buffer: Custom protocol

Node-RED Packages Required:

node-red-contrib-postgresql

2. Test Data Generator Flow

Purpose: Generate random test data every 5 seconds

Configuration:

- Inject every 5 seconds
- Sequential `product_count` (1, 2, 3...)
- Random `weight` (0.5 to 10.0)

- Fixed `product_name` = "Widget_A"

PostgreSQL Database

Database Details

- **Database Name:** `checkweigher`
- **Table Name:** `sensor_data`
- **Host (internal):** `postgres_db`
- **Host (external):** `localhost`
- **Port:** `5432`
- **User:** `postgres`
- **Password:** `yourpassword`

Table Structure

```
sql
CREATE TABLE sensor_data (
  id SERIAL PRIMARY KEY,
  product_name VARCHAR(255),
  product_count INTEGER,
  weight FLOAT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Column	Type	Description
id	SERIAL	Auto-increment primary key
product_name	VARCHAR(255)	Product identifier
product_count	INTEGER	Sequential count
weight	FLOAT	Product weight in kg
created_at	TIMESTAMP	Auto-generated timestamp

Access pgAdmin

1. Open: `http://localhost:5050`
 2. Login: `admin@admin.com` / `admin`
 3. Add Server:
 - Host: `postgres_db`
 - Port: `5432`
 - Database: `checkweigher`
 - User: `postgres`
 - Password: `yourpassword`
-

Appsmith Dashboard

Access URL

`http://localhost:8080`

Dashboard Components

1. Header

- **Widget:** Text
- **Content:** "PICO AUTOMATION"
- **Style:** Bold, 1.875rem, Center aligned

2. All Data Table (STATS)

- **Widget:** Table
- **Data Source:** `{{GetAllData.data}}`
- **Columns:** id, product_name, product_count, weight, created_at
- **Features:** Search, Sort, Pagination, Download

3. Statistics Table (ALL_DATA)

- **Widget:** Table
- **Data Source:** `{{GetStats.data}}`
- **Columns:**

- product_name
- total_records
- min_weight
- max_weight
- avg_weight
- current_count

4. Weight Chart (Chart1)

- **Widget:** Chart (Line)
- **Data Source:** `{{GetAllData.data.map(row => ({x: row.created_at, y: row.weight}}))}}`
- **X-Axis:** created_at (timestamp)
- **Y-Axis:** weight (kg)
- **Series Name:** "2023"

Database Queries

Query 1: GetAllData

sql

```
SELECT * FROM sensor_data ORDER BY created_at DESC;
```

- **Run on:** Page load
- **Refresh:** Every 5 seconds

Query 2: GetLatestData

sql

```
SELECT * FROM sensor_data ORDER BY created_at DESC LIMIT 1;
```

- **Run on:** Page load
- **Refresh:** Every 5 seconds

Query 3: GetStats

sql


```
SELECT
    product_name,
    COUNT(*) as total_records,
    MIN(weight) as min_weight,
    MAX(weight) as max_weight,
    AVG(weight) as avg_weight,
    MAX(product_count) as current_count
FROM sensor_data
GROUP BY product_name;
```

- **Run on:** Page load
- **Refresh:** Every 5 seconds

PostgreSQL Connection in Appsmith

- **Datasource Name:** PICO Chechweigher
 - **Plugin:** postgres-plugin
 - **Host:** `postgres` (service name from docker-compose)
 - **Port:** `5432`
 - **Database:** `checkweigher`
 - **Username:** `postgres`
 - **Password:** `yourpassword`
 - **SSL:** Disabled
-

Issues Faced & Solutions

Issue 1: WSL Shutdown

Problem: Docker containers stopped unexpectedly

Cause: Windows automatically shut down WSL when idle

Solution:

```
bash

wsl --shutdown

wsl
docker compose up -d
```

Prevention: For production, use dedicated Linux server instead of WSL

Issue 2: PostgreSQL Container Name Confusion

Problem: Couldn't connect to PostgreSQL from other containers

Cause: Using `container_name` instead of `service name`

Solution: Use service name from docker-compose.yml

- ✗ Wrong: `postgres_db` (container name)
 - ✓ Correct: `postgres` (service name)
-

Issue 3: Appsmith PostgreSQL Connection Failed

Problem: Appsmith couldn't connect to PostgreSQL

Cause: Used container name instead of service name

Solution: Changed host from `postgres_db` to `postgres`

Issue 4: Node-RED PostgreSQL Node Type

Problem: `pgconfig` and `pgquery` nodes not recognized

Cause: Wrong package name

Solution: Install `node-red-contrib-postgresql` package

Node Types: `postgresqlConfig` and `postgresql`

Issue 5: PostgreSQL SSL Error

Problem: "The server does not support SSL connections"

Cause: SSL config value was string `"disable"` instead of boolean

Solution: Changed `ssl: "disable"` to `ssl: false`

Issue 6: Mustache Template Rendering Error

Problem: Query syntax error - raw template showing in SQL

Cause: Mustache templates not rendering properly

Solution: Use `msg.query` instead of template field:

```
javascript
```

```
msg.query = "INSERT INTO sensor_data (product_name, product_count, weight)  
VALUES (' + data.product_name + ', ' + data.product_count + ', ' + data.weight + ');";
```

Issue 7: Parse TCP Data - Unrecognized Format

Problem: Inject node sending JSON but parser failing

Cause: Inject sends object, not string

Solution: Added check for object type first:

```
javascript
```

```
if (typeof msg.payload === 'object' && msg.payload !== null) {  
  parsed = msg.payload;  
}
```

Issue 8: init.sql Permission

Problem: PostgreSQL couldn't read init.sql

Cause: File had read-only permissions

Solution:

```
bash
```

```
chmod 666 ~/IIOT/init.sql
```

✅ Testing & Verification

Test 1: PostgreSQL Data Insert

Step 1: Click "Test Inject" in Node-RED

Step 2: Check Debug sidebar - should show inserted row

Step 3: In pgAdmin, run:

```
sql
```

```
SELECT * FROM sensor_data;
```

Expected: New row with test data

Test 2: Continuous Data Generation

Step 1: Deploy Node-RED flow with auto-inject

Step 2: Wait 10 seconds

Step 3: Check pgAdmin - should see multiple rows

Expected: New row every 5 seconds with:

- Sequential product_count (1, 2, 3...)
 - Random weight (0.5 to 10.0)
-

Test 3: Appsmith Dashboard Auto-Refresh

Step 1: Open Appsmith dashboard

Step 2: Watch the tables and chart

Expected: Data updates every 5 seconds automatically

Test 4: Container Network Communication

Step 1: Test from inside container

```
bash
```

```
docker exec -it appsmith ping postgres_db
```

Expected: Successful ping responses

Key Learnings

1. **Docker Networks:** Containers use service names to communicate, not IPs
2. **Port Mapping:** `(host:container)` format (e.g., `(8080:80)`)
3. **WSL Limitations:** Not suitable for production - use dedicated servers
4. **Node-RED Packages:** Always check exact package and node type names
5. **PostgreSQL Security:** Use parameterized queries to prevent SQL injection
6. **Appsmith Auto-Refresh:** Set query refresh intervals for real-time dashboards

Complete File List

```
project/
├── docker-compose.yml    # Container orchestration
├── init.sql              # Database initialization
├── nodered-flows/
│   ├── tcp-to-postgres.json # Main data flow
│   └── test-generator.json  # Test data generator
├── appsmith-export/
│   ├── application.json    # App configuration
│   ├── datasources/
│   │   └── PICO-Checkweigher.json
│   ├── pages/
│   │   ├── Dashboard/
│   │   │   ├── queries/
│   │   │   │   ├── GetAllData/
│   │   │   │   ├── GetLatestData/
│   │   │   │   └── GetStats/
│   │   └── widgets/
│   │       ├── STATS.json
│   │       ├── ALL_DATA.json
│   │       ├── Chart1.json
│   │       └── Text1.json
│   └── theme.json
```

Next Steps for Production

1. Security:

- Change default passwords
- Enable SSL for PostgreSQL
- Add authentication to Node-RED
- Use environment variables for secrets

2. Performance:

- Add database indexes on frequently queried columns
- Implement data archiving (move old data to archive table)

- Optimize Appsmith queries

3. **Monitoring:**

- Add error alerts in Node-RED
- Set up database backup schedule
- Monitor container health

4. **Scalability:**

- Move to dedicated Linux server
 - Add load balancer for high traffic
 - Consider PostgreSQL replication
-

Support & Maintenance

Access URLs:

- Node-RED: `http://localhost:1880`
- Appsmith: `http://localhost:8080`
- pgAdmin: `http://localhost:5050`

Restart All Services:

```
bash
docker compose restart
```

View Logs:

```
bash
docker logs node_red
docker logs postgres_db
docker logs appsmith
```

Backup Database:

```
bash
docker exec postgres_db pg_dump -U postgres checkweigher > backup.sql
```

End of Documentation